

fraud-mlops-pipeline

An end-to-end MLOps system for credit-card fraud detection
that detects drift in its own inputs and retrains itself

Adrián March Ramón

30 August 2026

Abstract

This paper documents the design, construction and verification of a production-shaped machine learning system for credit-card fraud detection, built over eleven weeks in nine gated phases and 142 commits. The system versions its data, tracks and gates its own model promotions, serves predictions over HTTP, ships itself through CI/CD to a public URL, orchestrates its own retraining on a schedule, and closes a monitoring loop that detects distributional drift in production traffic and triggers retraining without human intervention — a loop observed running end to end on two separate occasions. The fraud model itself is deliberately ordinary; the engineering around it is the subject. Every quantitative claim below is traceable to a commit, a decision record, a tracked experiment run, a workflow run or a live HTTP call, and the paper is explicit about what the system does *not* demonstrate: it has no real users, receives no ground-truth labels, and has never promoted a retrained model.

Contents

1	Executive summary	4
2	Introduction: the problem, and why the system rather than the model	5
2.1	Fraud as a needle-in-a-haystack problem	5
2.2	The cost asymmetry	5
2.3	The thesis: a good-enough model inside an excellent system	5
2.4	What this paper does not claim	6
3	System architecture	7
3.1	Two clocks on one infrastructure	7
3.2	Component map	7
3.3	The stack, and why each piece	8
3.4	Repository layout and the conventions that held	8
4	Methodology: how the project was run	10
4.1	Nine gated phases	10
4.2	The decision-record discipline	10
4.3	The verification standard	10
5	The build, phase by phase	12
5.1	Phase 0 — Foundations (17 commits)	12
5.2	Phase 1 — A versioned, validated data pipeline (18 commits)	12
5.3	Phase 2 — Training and experiment tracking (20 commits)	13
5.4	Phase 3 — Model Registry and packaging (12 commits)	13
5.5	Phase 4 — The inference API (12 commits)	14
5.6	Phase 5 — Containerization (10 commits)	14
5.7	Phase 6 — CI/CD (9 commits)	15
5.8	Phase 7 — Orchestration (13 commits)	15
5.9	Phase 8 — Monitoring and the closed loop (18 commits)	16
5.10	Phase 9 — Deployment and presentation (13 commits)	16
6	Design decisions in depth	18
6.1	The cost asymmetry, and everything downstream of it	18
6.2	Packaging the model with its preprocessing and threshold	18
6.3	Aliases over stages, and a promotion gate that refuses ties	18
6.4	The prediction log as a cross-phase contract	19
6.5	Data drift, not concept drift — and why the distinction is forced	19
6.6	Bundling the model for a registry-less deployment	19
7	Results	20
7.1	Model performance	20
7.2	The threshold trade, quantified	20
7.3	The closed loop, observed	20
7.4	System-level results	21
8	What went wrong	24
8.1	A deliberate failure: proving branch protection	24
8.2	An accidental failure: a commit split that broke CI	24
8.3	Two decision records disproved by their own verification	24
8.4	A failed flow run, and the fix it forced	25
8.5	The Python 3.14 repin	25

8.6	The cold-start corrections	25
8.7	A statistical test the repository named wrongly	25
8.8	What the record demonstrates	26
9	Limitations and future work	27
10	Lessons learned	28
10.1	Technical	28
10.2	Personal	28
11	Conclusion	30
12	Evidence index	31
A	The decision records	32
B	Reproducing the results	34

1 Executive summary

Most machine learning projects that reach a portfolio stop at the model. This one treats the model as the least interesting component and builds the system around it: the parts that make a model survivable in production rather than merely accurate on a test split once.

The problem is credit-card fraud detection on a public dataset of 284,807 transactions of which 492 — 0.172749% — are fraudulent. That imbalance makes accuracy meaningless and makes the choice of metric, threshold and error cost the substance of the modelling work rather than a footnote to it.

What was built, in eleven and a half weeks between 11 June and 30 August 2026:

- A **versioned, validated data pipeline** — DVC-tracked inputs, Pandera schema contracts enforced at both boundaries, a four-stage reproducible DAG.
- **Tracked training** over 24 experiment runs, with the winning model selected on PR-AUC and its operating threshold derived from an explicit business cost model.
- A **Model Registry** in which preprocessing, classifier and decision threshold travel as one artifact, and where promotion happens only when a candidate is strictly better than the model currently in production.
- A **typed inference API** that accepts raw transactions, logs every prediction, and resolves its model by role rather than by version number.
- **Containers, CI/CD and a public deployment** — a protected `main` that refuses any change failing lint, types, tests or a model-quality gate; an image published to GHCR on every merge; and that exact image serving a public URL.
- A **closed drift-to-retrain loop** — shifted traffic reaches the live API, Evidently measures the shift against a DVC-versioned reference, an alert fires, and a new model is trained and registered with no human in the path.

The system's headline results: **PR-AUC 0.876**, recall 0.888, precision 0.554, at a decision threshold of 0.03 chosen by minimising expected business cost rather than by maximising F1. The retraining loop was observed firing end to end twice, on 29 August 2026 at 16:55 and 20:48, each time producing a new registered model version in under a minute — 49 and 33 seconds respectively.

Two things this paper is careful not to claim. The loop has never promoted a retrained model, because retraining on unchanged data deterministically ties the incumbent and the promotion gate requires strictly better — seven of the eight registered versions sit unpromoted for exactly this reason, which is the gate working rather than the system failing. And the system measures *data* drift, never *concept* drift, because it never receives ground truth: in fraud, the label arrives days or weeks later with a chargeback.

Supporting the whole is a corpus of **44 architecture decision records**, one per decision, each written at the moment the decision was made, each carrying the alternatives that were rejected and the measurements that settled it. Several of them record the author being wrong and correcting it. That corpus, more than the code, is what this paper draws on.

2 Introduction: the problem, and why the system rather than the model

2.1 Fraud as a needle-in-a-haystack problem

The dataset is the Kaggle *Credit Card Fraud Detection* set (`mlg-ulb/creditcardfraud`): 284,807 transactions by 31 columns, all numeric. Thirty features — `Time`, `Amount`, and twenty-eight anonymised principal components `V1–V28` — and a binary target `Class`. Verified against the file itself during exploratory analysis and recorded in ADR 0003 (*dataset choice and known limitations*): no missing values, 1,081 exact duplicate rows, and a fraud rate of **0.172749%**.

The validation stage re-derives those figures on every pipeline run and writes them to a tracked report:

```
reports/validation.json
{ "n_rows": 284807, "n_fraud": 492, "fraud_rate": 0.001727, "status": "passed" }
```

At that prevalence, accuracy is a trap rather than a metric. A classifier that predicts “not fraud” unconditionally achieves 99.83% accuracy and catches nothing. This is not a subtlety to be mentioned and moved past; it determines every downstream choice in the project, and ADR 0001 (*business metric*) was written before any model existed precisely to fix that choice in advance of the temptation to report a flattering number.

2.2 The cost asymmetry

What makes fraud a business problem rather than a statistics exercise is that the two ways of being wrong cost very different amounts. ADR 0002 (*false-negative vs false-positive cost asymmetry*) records an explicitly *illustrative* model:

Error	Illustrative unit cost	Basis
False negative	$\approx$ €122 (mean fraud amount) + $\approx$ €15 handling $\approx$ €137	direct loss + operations
False positive	$\approx$ €3–5	support contact, friction, no direct loss

The ratio that follows — on the order of **34:1** — is the single most consequential number in the project, and the record is emphatic that it is not measured business data. The first figure comes from the dataset’s own amount distribution plus an operations estimate; the second is the midpoint of a plausible range. Every decision resting on it inherits that caveat, and both figures live in `params.yaml` as versioned parameters rather than as constants in code, so that real numbers can replace them without touching a line of logic.

Section 6.1 traces how that one ratio propagates through five later phases.

2.3 The thesis: a good-enough model inside an excellent system

The project’s governing constraint, fixed at Phase 0 and never relaxed, is that effort spent chasing additional PR-AUC points is effort not spent on the system. A model that scores well and cannot be reproduced, versioned, deployed, monitored or retrained is a worse engineering artifact than a mediocre model that can be.

This is not merely a stated preference; it is visible in where the 142 commits went. Of roughly 4,900 lines of Python across `src/`, `pipelines/`, `scripts/` and `tests/`, the modelling core — `train.py` and `evaluate.py` — accounts for 685 lines. The registry and packaging module alone is 797. The test suite is 1,216. The decision records run to 3,918 lines, more than the production source they explain.

2.4 What this paper does not claim

The system has no real users. It receives no ground-truth labels, and therefore cannot measure model *performance* decay, only input distribution change. No retrained model has ever been promoted. Its data remote is a local filesystem store, so the pipeline is reproducible on the author’s machine and not, without a Kaggle download, on anyone else’s. These are stated here rather than buried in a limitations section at the end, because a reader is entitled to know the shape of the claim before reading thirty pages that support it.

3 System architecture

3.1 Two clocks on one infrastructure

The system runs two loops at radically different frequencies over the same components.

The **prediction path** operates in milliseconds: an HTTP request arrives, Pydantic validates it, the packaged model applies its own preprocessing and threshold, a prediction is returned, and the input and output are appended to a log. Measured against the public deployment, this path completes in approximately 0.3 seconds end to end including network transit.

The **model-lifecycle path** operates over days: the accumulated prediction log is compared against a frozen reference distribution, drift is quantified, an alert fires, retraining is triggered, a candidate is evaluated against the incumbent, and the production alias moves only if the candidate wins.

The two paths share the prediction log, and that sharing is the architectural hinge of the entire system. The fast path writes it as a side effect; the slow path consumes it as its only view of production reality. Everything else — the registry, the orchestrator, the container topology — exists to make that hinge reliable.

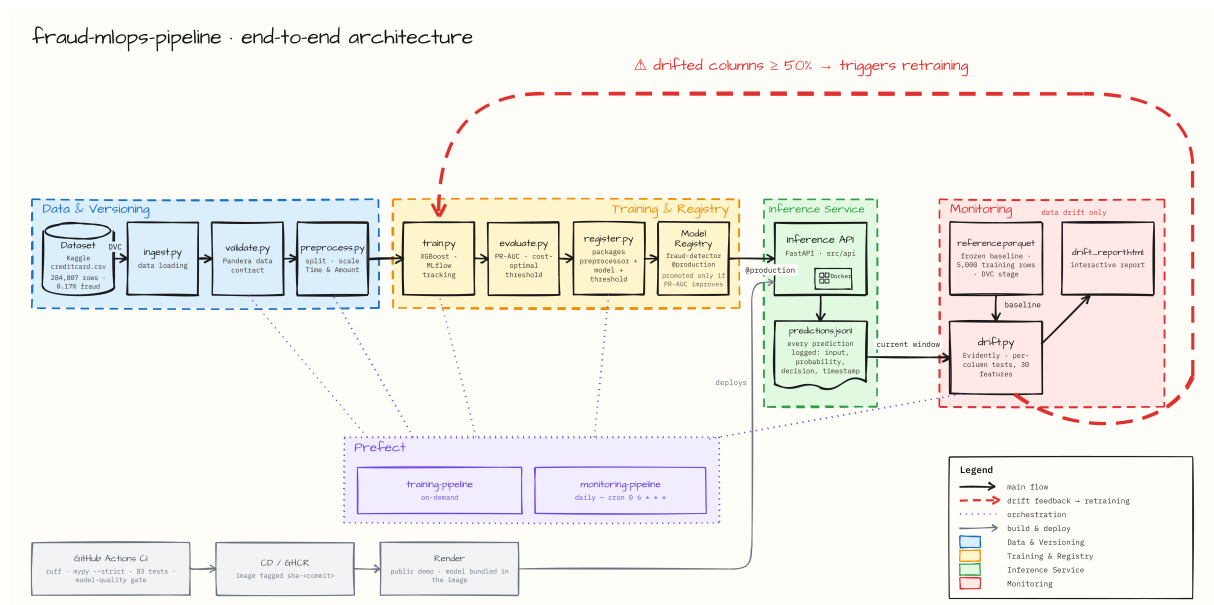


Figure 1: System architecture. Solid black is the main data flow; the thick red dashed arc is the drift feedback path that triggers retraining; violet dotted lines are Prefect orchestration; the grey band at the bottom is build and deploy. Reproduced from docs/images/architecture.png.

3.2 Component map

- **Data & versioning** — `ingest.py` loads the DVC-tracked CSV; `validate.py` enforces a Pandera contract; `preprocess.py` splits and scales.
- **Training & registry** — `train.py` fits and logs to MLflow; `evaluate.py` computes metrics and derives the cost-optimal threshold; `register.py` packages and promotes.
- **Inference** — `src/api/` serves three endpoints and appends every prediction to `logs/predictions.jsonl`.
- **Monitoring** — `reference.py` freezes a baseline as a DVC stage output; `drift.py` runs the Evidently comparison and emits both a verdict and an HTML report.

- **Orchestration** — `pipelines/` holds two Prefect flows and the deployment server, deliberately separate from `src/`.

A structural rule holds throughout: **nothing under `src/` imports `Prefect`**. Business logic knows nothing about how it is invoked, which is what allows the same `detect_drift()` to be called from a test, from a container, and from an orchestrated flow with identical behaviour.

3.3 The stack, and why each piece

ADR 0004 (*technology stack summary*) fixes the stack at Phase 0 with evidence for each choice. Three of them diverge from the reference roadmap the project was built against, and the divergences are more informative than the agreements.

Concern	Choice	Rationale
Environment	<code>uv</code>	One lockfile, no manual virtualenvs; <code>uv.lock</code> committed so CI and clones resolve identically.
Lint & format	<code>ruff</code> alone	<i>Divergence:</i> the roadmap prescribes <code>ruff</code> + <code>black</code> . <code>ruff format</code> subsumes <code>black</code> , leaving one tool.
Data versioning	DVC	Data never enters Git; only small pointer files do.
Data validation	Pandera	Schema as an executable contract at both data boundaries.
Modelling	scikit-learn / XGBoost	Baseline first, then a gradient-boosted challenger.
Tracking & registry	MLflow, SQLite backend	SQLite from day one because the Registry requires a database backend; choosing it at Phase 2 avoided a migration at Phase 3.
Serving	FastAPI + Pydantic	Typed request boundary and self-generating interactive docs.
Containers	Docker + Compose	Multi-stage, non-root, health-checked.
CI/CD	GitHub Actions + GHCR	Verification and delivery split into two workflows.
Orchestration	Prefect	<i>Divergence:</i> the roadmap offers “Prefect or ZenML”. Prefect chosen; the stale “to be decided” label survived in the architecture diagram until Phase 9.
Monitoring	Evidently 0.7.21	Column-level statistical drift tests plus a self-contained HTML report.
Deployment	Render, free tier	Lightweight PaaS. No Kubernetes — a deliberate scale-appropriate choice, not an omission.

A fourth divergence is more instructive still: the project initially pinned **Python 3.14**, and Phase 2 was forced to repin to 3.12 because MLflow’s UI server imports `importlib.abc.Traversable`, removed in 3.14. ADR 0004 () had itself warned that a very new interpreter risks the ecosystem lagging behind it. Section 8.5 returns to this.

3.4 Repository layout and the conventions that held

<code>src/</code>	all production code, importable package
<code>data/</code>	ingest, validate (Pandera), preprocess
<code>models/</code>	train, evaluate, register (MLflow)
<code>api/</code>	FastAPI app, Pydantic schemas, prediction logic
<code>monitoring/</code>	drift detection (Evidently), reference builder
<code>config.py</code>	centralized configuration - nothing hardcoded elsewhere
<code>pipelines/</code>	Prefect flows, kept separate from <code>src/</code>
<code>scripts/</code>	maintenance entry points (drift simulation, model export)
<code>tests/</code>	69 test functions, 83 cases under parametrisation
<code>deploy/model/</code>	the exported model that ships inside the public image
<code>docs/decisions/</code>	44 architecture decision records
<code>docker/</code>	Dockerfile (multi-stage), <code>docker-compose.yml</code>
<code>data/</code>	DVC-managed, never in Git

Four conventions were declared at Phase 0 and enforced mechanically thereafter: strict typing (`mypy -strict`, extended from `src/` to `pipelines/` in Phase 7 and to `scripts/` in Phase 8); structured logging and never `print()` inside `src/`; a custom exception hierarchy rooted at `FraudPipelineError` with no bare `except`; and centralised configuration in `src/config.py` with versioned hyperparameters in `params.yaml`.

The convention with the most visible consequence is the last one. Because `src/config.py` is a declared dependency of four DVC stages, every addition to it marks those stages stale even when the addition is inert. The repository contains five commits whose only purpose is to reconcile that drift (70726e0, d157120, ef92c36, a8b1819, 1b06ec2), each recording that no output hash moved. That is the visible cost of a single configuration surface, paid deliberately.

4 Methodology: how the project was run

4.1 Nine gated phases

The project was executed as nine sequential phases, each with a written Definition of Done, each closed by an annotated Git tag pointing at a commit whose CI had passed. A phase was not started until the previous one’s checklist was satisfied.

Phase	Milestone	Commits	Tagged
0	Repository, environment, EDA, decision log	17	2026-07-09
1	Versioned data pipeline (DVC + Pandera)	18	2026-07-12
2	Training and experiment tracking (MLflow)	20	2026-07-15
3	Model Registry and packaging	12	2026-08-22
4	Inference API (FastAPI)	12	2026-08-23
5	Containerization (Docker)	10	2026-08-23
6	CI/CD (GitHub Actions)	9	2026-08-24
7	Orchestration (Prefect)	13	2026-08-28
8	Monitoring, drift and the closed loop	18	2026-08-29
9	Deployment, definitive README, demo	13	2026-08-30

One feature of that table deserves comment: the later phases compress sharply — five of them inside nine days — because the foundations laid in Phases 0–2 made each subsequent layer mostly a matter of wiring rather than invention. Phase 5 in particular took ten commits to containerise a system that had been built, from the first line, to be containerisable.

4.2 The decision-record discipline

Every non-obvious decision was written down as it was made, in a numbered record following a fixed template: **Context** → **Decision** → **Alternatives considered** → **Justification** → **Trade-offs / consequences**. Forty-four records exist, numbered 0001–0044 and mapped to the phase that produced them (Appendix A).

Two properties make the corpus useful rather than ceremonial. First, *alternatives are mandatory*: a record that lists no rejected option is a record of a decision that was never actually made. Second, *justification must be measured*. The index states the evidence class for each range explicitly — notebook cells for 0001–0010, tracked run identifiers for 0011–0014, registered versions and the commands that produced them for 0015–0018, live HTTP calls for 0019–0021, built images and running containers for 0022–0025, workflow runs and published packages for 0026–0030, and so on through the live Prefect flow runs and Evidently probes of the later phases.

The corpus also records its own errors. Two records were corrected in place after their own verification disproved them (Section 8.3), and one — ADR 0018 (*registry state at Phase 3 closure*) — is explicitly marked as a dated snapshot that goes out of date by design.

4.3 The verification standard

Three habits distinguish the project’s verification from the more common practice of running a command and declaring success.

Predictions were stated before runs. Before both closed-loop demonstrations, the expected outcome was written down — training is deterministic on unchanged data and the promotion gate requires strictly better, therefore a new version should be created and refused promotion — so that the observed result could be *checked* rather than rationalised afterwards. Both times it matched, to the last digit of PR-AUC.

Tests were mutation-tested. A green suite proves nothing about whether it would catch a regression. In Phase 8, corrupting the metric identifier that `drift.py` uses to extract Evidently’s aggregate turned 11 passing tests into 4 failures, and restoring it returned them to green. In Phase 9, three mutations were applied to the new model-loading branch; two were caught and one was not, which exposed a genuine hole in the tests that was then closed (Section 5.10).

Closure re-queried live state rather than citing earlier work. Each phase closure read the current state of the Prefect database, the MLflow registry or the GitHub API afresh, rather than repeating figures from the interaction that produced them. Phase 8’s closure went further and re-ran the entire loop, because one of its three evidence sources had been deliberately reset in the interim.

5 The build, phase by phase

5.1 Phase 0 — Foundations (17 commits)

Phase 0 produced no machine learning at all. It produced the conditions under which the rest could be built: an environment managed entirely by `uv` with a committed lockfile, `ruff` configured as both linter and formatter, `pre-commit` hooks including a large-file guard, a `Makefile` as the canonical interface, an exploratory notebook, and the first four decision records.

The notebook is the source of every dataset figure quoted in this paper, and it was quarantined to exploration from the start — no production code has ever imported from it. The decisions it supported were fixed immediately: the primary metric (ADR 0001 ()), the cost asymmetry (ADR 0002 ()), the dataset and its known limitations (ADR 0003 ()), and the full stack (ADR 0004 ()).

Fixing the metric before writing a model is the phase’s most consequential act. ADR 0001 () rejects accuracy (“99.83% is achievable by predicting no fraud at all”), rejects ROC-AUC as primary (“under severe imbalance the false-positive rate is diluted by the huge negative class, so ROC-AUC looks deceptively optimistic”), and rejects F1 at the default threshold (“0.5 is meaningless at this prevalence”). It also states, before any of the machinery existed to enforce it, that *the decision threshold is a tuned, versioned part of the model artifact, not a hardcoded 0.5 in the API*. Phase 3 delivered on that sentence three months later.

The phase also pinned Python 3.14, a decision Phase 2 was forced to reverse (Section 8.5).

5.2 Phase 1 — A versioned, validated data pipeline (18 commits)

Phase 1 established two properties the rest of the project depends on: that the data is *versioned* and that it is *validated*.

DVC was initialised, a remote configured, and the raw CSV tracked — with only the small `.dvc` pointer entering Git. ADR 0005 (*DVC remote: local filesystem store*) records the remote as a local directory outside the repository, explicitly interim, and Section 9 returns to the reproducibility cost that choice imposes on third parties.

Validation was implemented as a Pandera contract applied at *both* data boundaries — raw input and preprocessing output. ADR 0010 (*Pandera contract*) fixes three settings and gives a reason for each: `strict=True` rejects any undeclared column; `lazy=True` accumulates every failure before raising, so a broken dataset reports all its problems at once rather than one per run; `coerce=True` attempts declared-type coercion first.

Three decisions in this phase are about leakage and skew, and all three pay off much later:

- ADR 0006 (*no data leakage*) — the scaler is fitted on the training split only, never on the full dataset.
- ADR 0007 (*scale Time and Amount only*) — V1–V28 are already PCA components and pass through untouched.
- ADR 0008 (*persist the fitted preprocessor*) — the fitted `ColumnTransformer` is serialised to `preprocessor.joblib` and treated as a first-class versioned artifact, with the explicit statement that “the future inference API (Phase 4) will load this exact object ... never reimplement the preprocessing”.

That last record is the origin of the project’s defence against training-serving skew. It was written in Phase 1 and consumed in Phase 3.

The phase closed with a two-stage DVC pipeline, `validate` → `preprocess`, and its first real execution (65f0c49).

5.3 Phase 2 — Training and experiment tracking (20 commits)

Phase 2 is where the modelling happens, and where the project’s largest single divergence from plan occurred.

MLflow was configured with a **SQLite backend from day one** rather than flat file storage. ADR 0011 () gives the reason plainly: the Model Registry, needed in Phase 3, requires a database backend, so choosing SQLite now avoided migrating the backend later. This is a recurring pattern in the project — paying a small cost early to avoid a migration that would otherwise land in the middle of a later phase.

Twenty-four runs were tracked in the **fraud-detection** experiment. The decisive ones:

Run	Model	Thr.	PR-AUC	Recall	Precision
db6ac47e	logistic regression	0.5	0.7159	0.9184	0.0610
f01f26b5	logistic regression + SMOTE	0.5	0.7249	0.9184	0.0580
9c635274	XGBoost	0.5	0.8760	0.8367	0.8723
7f5ef7b2	XGBoost	0.03	0.8760	0.8878	0.5541
f97d88a7	XGBoost, <code>n_estimators=200</code>	0.03	0.8655	0.8878	0.2949

Table 2: Selected runs from the MLflow **fraud-detection** experiment, read from the tracking database. The full experiment holds 24 runs.

The logistic-regression baseline is instructive precisely because it fails: recall 0.9184 sounds excellent until precision 0.0610 is read beside it. At that operating point roughly 94% of alerts are false, which is not a fraud system but a nuisance generator. ADR 0013 (*winning model*) promotes XGBoost on PR-AUC, 0.8760 against 0.7249.

Class imbalance is handled at two levels, and ADR 0012 (*imbalance strategy*) is careful about the interaction. Within training, class weighting and SMOTE are *mutually exclusive per run* — enabling SMOTE explicitly switches weighting off, verified by asserting that a model built with `class_weight="balanced"` reports `class_weight=None` after passing through the SMOTE branch. XGBoost instead uses `scale_pos_weight = 577.29` and therefore *must* pair with **resampling: none**, or the correction would be applied twice. Post-hoc threshold adjustment is orthogonal and always applies on top.

The threshold itself is the phase’s headline decision. ADR 0014 () adopts **0.03**, the cost-optimal operating point measured over the held-out test split, against alternatives of 0.5 (“provisional”) and the F1-optimal 0.9921. Expected business cost under ADR 0002 ()’s model: €1,787 at 0.03, €2,240 at 0.5, €2,885 at F1-optimal. Section 7.2 quantifies what that costs in precision.

Finally, the DVC pipeline was extended from data into the model (`498e5cc`), so `dvc repro` now rebuilds `raw` → `validate` → `preprocess` → `train` and re-runs only what changed.

5.4 Phase 3 — Model Registry and packaging (12 commits)

Phase 3 answers a question Phases 1 and 2 deliberately deferred: what, exactly, is the deployable unit?

The answer is `FraudModel`, an `mlflow.pyfunc.PythonModel` that carries the Phase 1 preprocessor, the Phase 2 classifier and the versioned threshold as a single artifact. ADR 0015 (*packaged model contract*) fixes its interface: **raw transactions in, probability and decision out**. The API never scales a column or applies a threshold.

The record diverges from the reference material on one axis and explains why. The reference shows a three-argument constructor holding both fitted objects as instance state; the implementation takes only the threshold and receives preprocessor and classifier through `context.artifacts`. The alternative was tested rather than assumed — a fitted `ColumnTransformer` does round-trip through cloudpickle — and rejected because artifact injection rebuilds each component through

its own MLflow flavour, with the requirements MLflow recorded for it, instead of collapsing a scikit-learn pipeline and an XGBoost booster into one opaque blob.

Promotion is governed by ADR 0016 (*promotion quality gate*), which fixes four rules:

- **Aliases, never stages.** One alias, `@production`, moved with `set_registered_model_alias()`. No stage API appears anywhere in `src/`.
- **PR-AUC is the sole criterion**, read from the candidate's own version tag and compared against the tag of the incumbent.
- **Strictly better wins.** Equal does not promote.
- **No production model yet is a valid state**, not an error — the first candidate is promoted unconditionally.

The third rule is the one that produces the project's most counter-intuitive result, and Section 7.3 defends it.

5.5 Phase 4 — The inference API (12 commits)

Phase 4 puts the registered model behind HTTP: three endpoints, a Pydantic request boundary, and a prediction log.

ADR 0020 (*API structure*) separates concerns strictly — `predict.py` holds inference and knows nothing about HTTP (no FastAPI import, no status codes), while `main.py` owns the routes and translates failures into status codes. The model is loaded *once* at startup into `app.state`, because unpacking the artifact costs far more than scoring does.

The same record makes a decision that looks like a bug and is not: **a failed model load is logged and swallowed**, the service starts anyway, `/health` reports the degraded state, and the other two endpoints answer 503. The reasoning is that an orchestrator can act on an unhealthy instance it can inspect, but not on a process that crash-loops with no surface. Phase 5 then had to work around the consequence: `/health` returns HTTP 200 even with no model, so a bare `curl -f health` check would call a broken container healthy.

The phase's most far-sighted artifact is the prediction log. ADR 0021 (*prediction log as Phase 8's contract*) froze the filename, the record shape and the "input" key, stating explicitly that these are a contract with a phase that did not yet exist. Section 6.4 follows that thread to its conclusion.

5.6 Phase 5 — Containerization (10 commits)

Ten commits, of which two are bug fixes for failures that only appear inside a container — which is the phase's real lesson.

ADR 0022 (*container image contract*) specifies a two-stage build on `python:3.12-slim` with `uv` copied from its own pinned image rather than installed by a piped script, dependencies resolved before source so that editing `src/` reuses the cached layer, a non-root `appuser`, and a health check that **matches the response body** rather than the status code.

That health-check decision was not theoretical. Commit 24d8a07 records the case it was designed for: MLflow 3 validates the `Host` header of every request as DNS-rebinding protection, and its default allowlist covers loopback but not container names — so the API's `http://mlflow:5000` was answered with 403 `Invalid Host header` while host-side training worked fine. The failure was invisible to a naive health check because `/health` and `/version` are exempt from that validation: the service reported healthy while being unusable for the registry calls that mattered.

Commit 854484a records the second: because `WORKDIR` creates `/app` as root and `COPY -chown` applies only to entries it writes, `appuser` could read everything and create nothing. `log_prediction()`'s `makedirs` raised `EACCES`, its `except OSError` branch swallowed it *by design*,

and every prediction was silently dropped while `/predict` still answered 200 — an endpoint that looked flawless over an empty log, which would have left Phase 8 with no baseline at all.

ADR 0024 (*environment-based tracking URI*) establishes the pattern that carries the project through two more phases: one variable, `MLFLOW_TRACKING_URI`, with a working local default, so the same `train.py` and `register.py` populate either a local SQLite store or a containerised MLflow without modification. The record names itself a precedent for “Phase 6’s CI and Phase 9’s deployment target” — and Phase 9 used it exactly as written.

5.7 Phase 6 — CI/CD (9 commits)

Phase 6 splits verification from delivery into two workflows with different triggers, different permissions and different concurrency policies.

`ci.yml` runs on every pull request: `uv sync -locked` (which fails loudly if the lockfile has drifted), then `ruff check`, `ruff format -check`, `mypy -strict`, then the full test suite — cheapest check first, so a misplaced import costs seconds rather than minutes. `cd.yml` runs only on pushes to `main` and publishes the image to GHCR tagged `latest`, `main` and `sha-<commit>`.

ADR 0026 () explains why CD deliberately re-runs none of CI’s checks: the only commits that reach `main` are those a protected pull request let through, and those are exactly the commits CI already validated. The trust chain is stated explicitly — model gate decides whether CI is green, branch protection decides whether a red CI can merge, merging is the only way to reach `main`, reaching `main` is the only thing that triggers CD — and its weak link identified: the protection rule, not the workflow.

The phase’s distinctive contribution is `tests/test_model_quality.py`, the **model-validation gate**. Passing code tests say nothing about whether the model still works; this test reads the PR-AUC that DVC versions in Git and fails the build if it has degraded below `MIN_PR_AUC = 0.75` (ADR 0028 ()).

And rather than assume branch protection worked, Phase 6 proved it by engineering a failure — Section 8.1.

5.8 Phase 7 — Orchestration (13 commits)

Phase 7 adds Prefect *alongside* DVC rather than instead of it. ADR 0033 (*orchestration design*) draws the line cleanly: `dvc.yaml` answers “what ran and with what result” (deterministic, cached, content-addressed); the flow answers “when it ran and how reliably” (schedule, retries, observability, event triggering). Both layers stay.

Two design rules keep the orchestration thin. Tasks call the Python functions directly, never `dvc repro`. And **one task wraps exactly one existing function** — no business logic is rewritten, reordered or reconfigured; the flow’s sole responsibility is sequencing. Retries are calibrated per stage rather than globally: validation is cheap and fails for transient reasons, so it retries three times; training costs minutes and usually fails deterministically, so it retries twice with a longer pause.

Two deployments were registered:

<code>training-pipeline/on-demand</code>	<code>manual, or triggered by another flow</code>
<code>monitoring-pipeline/daily</code>	<code>cron 0 6 * * *</code>

The monitoring flow calls `run_deployment()` with `timeout=0` — fire-and-forget, so a monitoring run submits a retrain and returns rather than holding its scheduled slot open for the minutes training takes.

Crucially, the flow was wired around a `detect_drift()` that was a **placeholder pinned to False** (3aea915). The loop was fully constructed and provably inert. Phase 8 changed the predicate and nothing else.

Phase 7 also produced a genuine debugging episode and a corrected decision record — Sections 8.3 and 8.4.

5.9 Phase 8 — Monitoring and the closed loop (18 commits)

Phase 8 is the centrepiece, and the phase where the reference material was least reliable.

The comparison has two sides. The **reference** is 5,000 rows drawn from the training split in raw feature space, built by a dedicated DVC stage so the baseline has a content hash and two runs a month apart answer the same question. ADR 0037 () explains both properties: training rows because that is the distribution the model actually learned, and raw space because the other side of the comparison — the API’s prediction log — is untransformed.

The **current window** is the prediction log itself, read through the contract Phase 4 froze.

ADR 0038 (*Evidently dependency and API*) records a finding that invalidated the reference material’s code sample outright: Evidently’s public API changed shape between generations, and the prescribed accessor

```
report.as_dict()["metrics"][0]["result"]["share_of_drifted_columns"]
```

does not exist in the installed 0.7.21 — `as_dict` is absent from the object entirely. Every access was therefore written against a probe of the actual release. The aggregate is located by matching a stable config type rather than by list index, because the preset is free to reorder its metrics between releases and an index would then read a per-column p-value as if it were the dataset verdict — silently, since both are numbers.

Two policy decisions sit on top of the measurement. `DRIFT_THRESHOLD = 0.5`: the share of drifted columns at or above which retraining fires. And `DRIFT_MIN_ROWS = 100`, below which the detector *declines to answer* rather than guess — measured, not assumed, because Evidently run on a 3-row current frame against a 500-row reference reports every column drifted. Without that floor, the first scheduled run would have retrained on the three smoke-test records already in the log.

Alerting is two-tiered (ADR 0039 ()): always to the log, and to a webhook when `DRIFT_WEBHOOK_URL` is configured. A webhook failure is caught and logged rather than raised, and the reasoning is specific — the function is called from inside a Prefect task with `retries=2`, so an escaping `ConnectionError` would fail the monitoring run, burn both retries, and prevent the retraining trigger that follows it from ever executing. A notification outage would become a model outage.

The phase ends with the loop observed running end to end, twice (Section 7.3).

5.10 Phase 9 — Deployment and presentation (13 commits)

Phase 9 had to solve a problem the reference material poses but does not resolve: it asks for both “the same image your CI already builds and publishes” *and* “the registered model inside the image”, and those are mutually exclusive as the repository stands.

The audit that opened the phase established why. **No trained model artifact is versioned anywhere** — `dvc.yaml`’s `train` stage declares only `reports/metrics.json` as an output, and `.gitignore` excludes `mlruns/`, `mlflow.db` and `models/`. The model existed in exactly one place: a git-ignored, DVC-untracked store on one machine. A GitHub runner has neither the dataset nor a Registry; neither does Render, which builds from the same repository. Anything the image is to contain must already be in Git.

The obvious workaround — copy the MLflow store into the image — was measured and rejected. The store records **absolute host paths** in four columns (`experiments.artifact_location`, `runs.artifact_uri`, `logged_models.artifact_location`, `model_versions.storage_location`),

so a copy resolves its registry rows and then chases artifacts at a path that does not exist in the container.

ADR 0042 (*bundled model*) therefore exports the `@production` artifact into `deploy/model/` — 38 files, 1.4 MB, byte-identical to the registry’s copy — and adds one environment-conditioned branch: with `MODEL_PATH` set the model is read from that directory and no Registry is contacted; unset, the alias resolves exactly as before. This is ADR 0024 ()’s pattern applied a third time, and it supersedes, *in scope only*, ADR 0022 ()’s rejection of baking a model into the image — a rejection that still stands for the Compose topology, where an alias still has something to indirect through.

Feasibility was proven before any of it was written: the exported artifact was loaded from a directory outside the repository, with `PYTHONPATH` empty and `MLFLOW_TRACKING_URI` pointed at an unreachable address, while an instrumented `joblib.load` recorded every path it opened. Exactly one call, inside the export directory — MLflow follows the relative path in `MLmodel`, never the absolute URI recorded beside it. The prediction matched the registry-loaded artifact to the last digit.

ADR 0043 (*Render deployment*) then deploys *that exact published image*, pinned by its immutable `sha-` tag, on Render’s free tier, with the health check pointed at `/model-info` rather than `/health` — because Render reads only status codes and `/health` answers 200 with no model loaded by design, the same trap ADR 0022 () avoided in the Dockerfile by matching the body.

The phase closed with a redrawn architecture diagram, a rewritten README, a 20-second introduction video, and ADR 0044 (*Phase 9 closure*), which records two Definition-of-Done items as *not* cleanly met: the video is 20 seconds against a criterion asking for two to three minutes, and the “key test” — that a stranger understands the project in two minutes — is left unclaimed, being the one criterion in nine phases the repository cannot evaluate about itself.

6 Design decisions in depth

Six decisions carried more weight than the rest, in the sense that reversing any of them would change the shape of the system rather than one of its details.

6.1 The cost asymmetry, and everything downstream of it

A single ratio recorded in week one propagates through five later phases, and tracing it is the clearest way to see the project as one design rather than nine.

Phase	Record	What the ratio determined
0	ADR 0002 ()	FN $\approx$ €137, FP $\approx$ €4 — a $\approx$ 34:1 asymmetry.
0	ADR 0001 ()	Therefore PR-AUC, not ROC-AUC or accuracy; recall prioritised, precision controlled.
2	ADR 0014 ()	Therefore threshold 0.03, minimising expected cost — not 0.5, not F1-optimal.
3	ADR 0016 ()	Therefore promotion is decided on PR-AUC alone, strictly better.
6	ADR 0028 ()	Therefore CI blocks any merge whose PR-AUC falls below 0.75.

Each step is a consequence of the one above it, and the chain terminates in an automated gate that blocks a merge. An assumption stated in prose in June became executable policy in August without anyone re-litigating it — and, equally important, the assumption remained visible: every record in the chain inherits ADR 0002 ()’s “illustrative, pending real business data” caveat, so a reader who disputes the ratio knows precisely which five decisions to revisit.

6.2 Packaging the model with its preprocessing and threshold

The single most valuable structural decision in the project is that the deployable artifact is *not* a model. It is a model, plus the fitted preprocessor that produced its training features, plus the decision threshold, as one versioned unit.

The failure this prevents is **training-serving skew**: the API reimplements feature engineering slightly differently from training — a different scaler, a column in a different order, a threshold someone hardcoded at 0.5 — and nothing ever raises an error. Predictions degrade silently, and because no exception is thrown, no alert fires and no test fails.

The defence spans three phases. ADR 0008 () (Phase 1) persists the fitted transformer instead of re-deriving it. ADR 0015 () (Phase 3) packages it with the classifier and threshold behind a `predict()` that takes raw transactions. ADR 0019 () (Phase 4) has the API pass `transaction.model_dump()` straight in. The result is that `src/api/predict.py` contains *no feature engineering whatsoever*: its 266 lines are alias resolution, bundle loading, error translation and a shape check on what the artifact returns. There is nowhere in the serving path for the training and serving transformations to diverge, because there is only one of them.

6.3 Aliases over stages, and a promotion gate that refuses ties

MLflow offers both a legacy stage API and aliases. The project uses one alias, `@production`, and no stage API appears anywhere in `src/`.

The consequence is that consumers name a *role*, never a version. The API resolves `models:/fraud-detector@production` once at startup; promoting a new version changes what it serves after a restart with no code change. That indirection is what makes the retraining loop meaningful — a loop that produced models nobody could route traffic to would be an expensive no-op.

The gate itself requires *strictly* better PR-AUC. Section 7.3 shows why that rule, which looks pedantic, produces the project’s most honest result.

6.4 The prediction log as a cross-phase contract

In Phase 4, ADR 0021 () froze three things about `logs/predictions.jsonl`: its path, its one-JSON-object-per-line format, and the key `"input"` holding the raw request payload. The record states outright that these constitute a contract with Phase 8, a phase that would not begin for another six days and whose implementation was entirely unwritten.

The contract also shaped decisions that look unrelated. ADR 0023 () rejected swapping the JSONL log for a logging database in the Compose topology, on the explicit ground that it would break a phase not yet written. The Dockerfile’s `chown` of `/app/logs` (854484a) exists because without it the log silently stayed empty — and the commit message says so, naming Phase 8 as what would have been damaged.

When Phase 8 arrived, `src/monitoring/drift.py` read the file unchanged. Its `_load_current()` lifts `record["input"]` per line and refuses to skip a malformed one, on the reasoning that silently dropping records would bias the very distribution being measured.

This is the clearest illustration of the project’s central method: *monitoring is not something you add later; it is something you leave room for.*

6.5 Data drift, not concept drift — and why the distinction is forced

The system measures $P(X)$, the input distribution. It does not measure $P(y | X)$, the relationship between features and outcome, and it cannot.

Concept drift requires ground truth. Nothing in this project ever receives it: the prediction log records the model’s own verdict, never the outcome. In fraud specifically, the truth arrives days or weeks later, when a chargeback is filed — the **label delay** problem. A system that claimed to measure concept drift here would be measuring nothing.

Two consequences follow, and the project accepts both. Input drift is the honest signal available, and it is also the *early* one — it moves before any label could have arrived. But it is a proxy: input distributions can shift without harming performance, and performance can decay while inputs look stable. The system will fire on the first and miss the second.

6.6 Bundling the model for a registry-less deployment

Phase 9 forced a decision that contradicted an earlier one, and the interesting part is how the contradiction was handled.

ADR 0022 () had rejected baking a model into the image, with a good reason: it ties one image to one model version and forces a rebuild on every promotion. ADR 0042 () does exactly that — but only for the public deployment, where no Registry exists for an alias to resolve against, so the indirection ADR 0022 () was protecting has nothing to indirect through.

The resolution was to supersede *in scope*, not wholesale: the record is marked as superseding one alternative of ADR 0022 (), ADR 0022 () gained a forward pointer so its rejection is not read as still absolute, and the Compose path was left untouched and verified untouched. A single environment variable separates the two worlds, and with it unset every workflow behaves exactly as it did in Phases 4–8.

7 Results

7.1 Model performance

Metric	Value
PR-AUC (primary)	0.8760
Recall	0.8878
Precision	0.5541
F1	0.6824

Table 3: Held-out test performance, from `reports/metrics.json`. This file is versioned in Git by DVC and read by CI on every pull request, so these figures are machine-checked rather than transcribed.

In business terms: the system catches **89% of fraud**, and roughly **45% of its alerts are false alarms**. Stated without context that precision looks poor. Under a 34:1 cost asymmetry it is the cheaper mistake, and the next section quantifies the trade exactly.

7.2 The threshold trade, quantified

Because the same XGBoost model was evaluated at two thresholds within the tracked experiment, the cost of the operating point can be read directly rather than argued:

Threshold	Recall	Precision	Expected cost
0.5 (default)	0.8367	0.8723	€2,240
0.03 (cost-optimal)	0.8878	0.5541	€1,787
0.9921 (F1-optimal)	—	—	€2,885

Moving from 0.5 to 0.03 **traded 31.8 points of precision for 5.1 points of recall**. On F1 that is a clear loss — 0.8542 down to 0.6824. Under the cost model it is a 20% reduction in expected loss, because each of those 5 recall points is worth roughly 34 false alarms.

This is the number that most clearly separates a metric-optimising project from a business-optimising one, and it is why ADR 0001 () was written before any model existed: had the project optimised F1 by default, it would have chosen a threshold of 0.9921 and been *worse* than doing nothing thoughtful at all.

7.3 The closed loop, observed

The loop was demonstrated twice, on 29 August 2026. The second run, executed during phase closure rather than cited from the interaction that produced the first, is reproduced from ADR 0041 ():

```
20:47:12 BEFORE Registry v1..v7, @production v1, pr_auc 0.875996
20:47:12 BEFORE logs/predictions.jsonl: 3 records
20:47:12 BEFORE detect_drift() -> False
20:47:47 INJECT "Drift check skipped: 3 rows, below the 100-row minimum"
20:47:47 INJECT make simulate-drift -> Accepted: 300/300 (0 failures)
20:47:47 INJECT log 3 -> 303; injected Amount mean 2910.20 (reference 87.86),
20:47:47 INJECT V1 mean 3.080 (reference 0.010)
20:47:54 TRIGGER monitoring-pipeline/daily -> flow run 'orchid-capybara'
20:48:02 Task 'Check drift' Drift detected? True
20:48:02 DRIFT ALERT: Significant drift detected. Triggering retraining.
20:48:02 flow run 'comical-turaco' created, source=deployment
20:48:11 Validate data 284807 rows, fraud rate 0.0017
```

```

20:48:12      Preprocess data      Completed
20:48:19      Train model      Completed
20:48:20      Register and promote  Completed
20:48:21      Flow 'comical-turaco' Finished in state Completed()
20:48:20  AFTER      fraud-detector v8 created, aliases=[], pr_auc 0.8759962787477742
                        @production -> v1 (unchanged)

```

Three observations, in ascending order of importance.

Thirty-three seconds elapsed between the injected batch being accepted (20:47:47) and a new registered model version existing (20:48:20), with no human input at any point. The first demonstration, earlier the same day, took 49 seconds on the same path. `comical-turaco` carries `deployment` provenance, meaning it was created by `run_deployment()` from inside the monitoring flow rather than launched by hand.

The False at 20:47:12 matters as much as the True at 20:48:02. It is not merely “no drift” — it is the minimum-row guard refusing to answer on three rows. Without it, the demonstration would not distinguish a working detector from one that reports drift on anything.

v8 was created and refused promotion, and this is the result the project is most willing to defend. Training is deterministic on unchanged data, so the retrained model tied the incumbent to the last digit of PR-AUC; the gate requires strictly better; therefore no promotion. **A gate that refuses an equal model is a gate that works.** Seven of the eight registered versions sit unpromoted for this reason.

The outcome was predicted in writing before the run, precisely so it could be checked rather than rationalised afterwards.

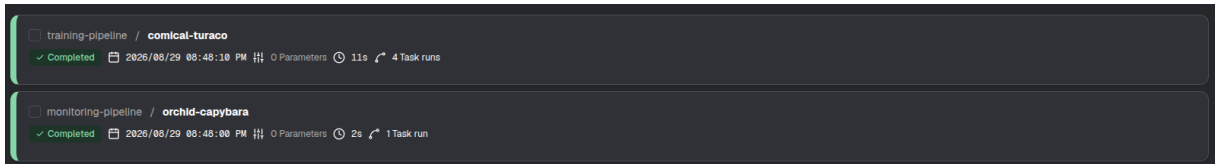


Figure 2: The loop, in the Prefect UI. `monitoring-pipeline/orchid-capybara` completed at 20:48:00 in two seconds with a single task; `training-pipeline/comical-turaco` began ten seconds later at 20:48:10 and ran four tasks in eleven. Nothing happened between them except the drift verdict.

7.4 System-level results

Dimension	Measured
Delivery	33 CI runs (31 success, 2 failure), 15 CD runs (15 success), 17 image tags in GHCR
History	142 commits, zero merge commits , 17 pull requests, 10 annotated tags
Quality gates	<code>ruff</code> , <code>ruff format -check</code> , <code>mypy -strict</code> over 25 files, 83 test cases, plus the PR-AUC gate
Clean clone	<code>git clone</code> → <code>make setup</code> → <code>make test</code> = 83 passed in 13.17s, with no dataset present
Public API	<code>/health</code> , <code>/docs</code> , <code>/model-info</code> , <code>/predict</code> all 200; ≈ 0.3 s warm
Cold start	111 s and 191 s measured after idle; still warm after 17 minutes
Footprint	199 MiB resident against Render’s 512 MB free-tier cap; image 0.77 GB

The zero-merge-commit figure is worth a sentence. Every one of the 15 merged pull requests

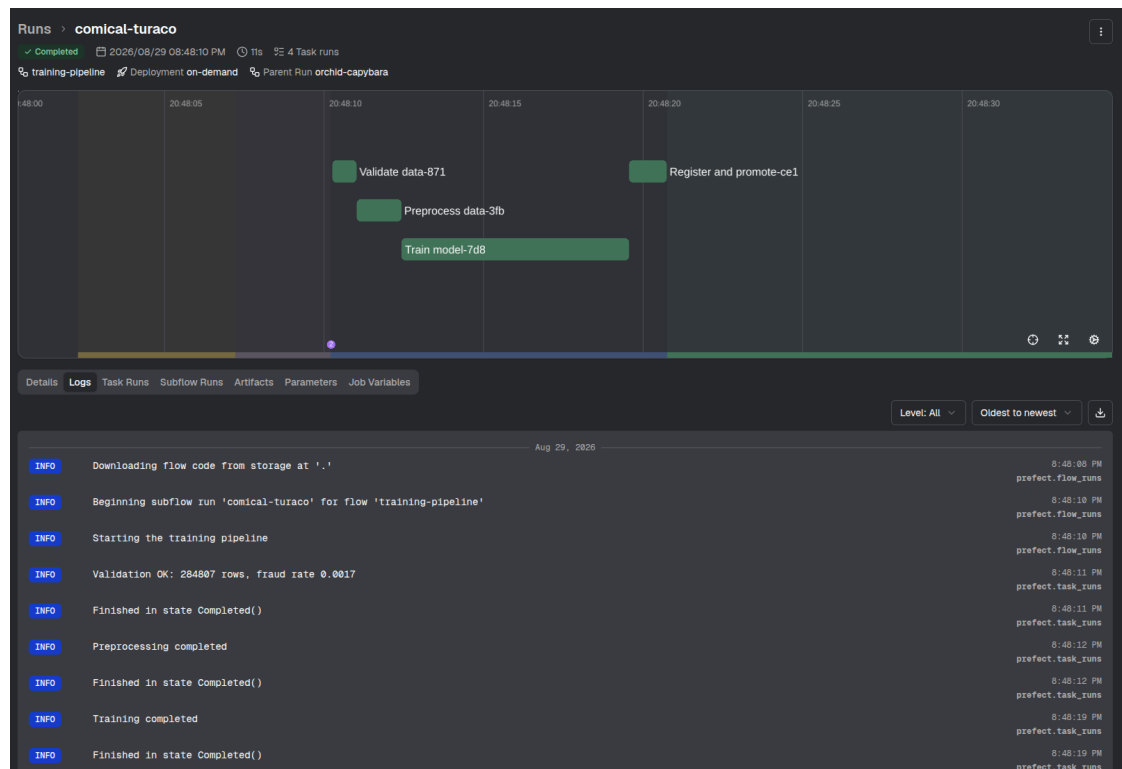


Figure 3: The retraining run in detail. The header establishes provenance — **Deployment on-demand** and **Parent Run orchid-capybara** — so this run was created by the monitoring flow rather than launched by hand. The four tasks (validate, preprocess, train, register) and the log line `Validation OK: 284807 rows, fraud rate 0.0017` at 8:48:11 correspond exactly to the transcript above.

was integrated by rebase, giving a linear history across 142 commits in which every commit is a self-contained change — which is what made the phase-by-phase reconstruction in this paper possible from `git log` alone.

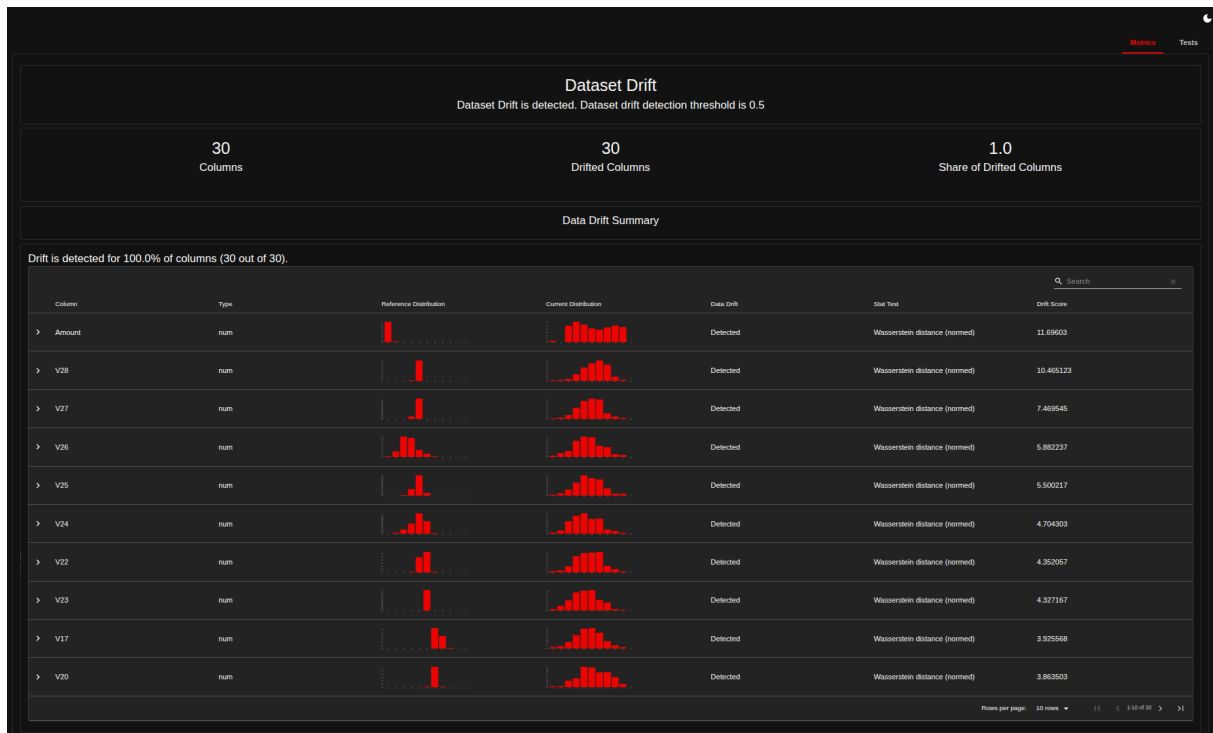


Figure 4: The Evidently report written by the same run. **30 of 30 columns drifted**, a share of **1.0** against the configured detection threshold of 0.5. Evidently selected the **Wasserstein distance (normed)** for every column, and **Amount** carries the largest drift score, 11.70. Capturing this figure is what exposed a documentation error corrected in Section 8.7. The paired sparklines are the reference and current distributions.

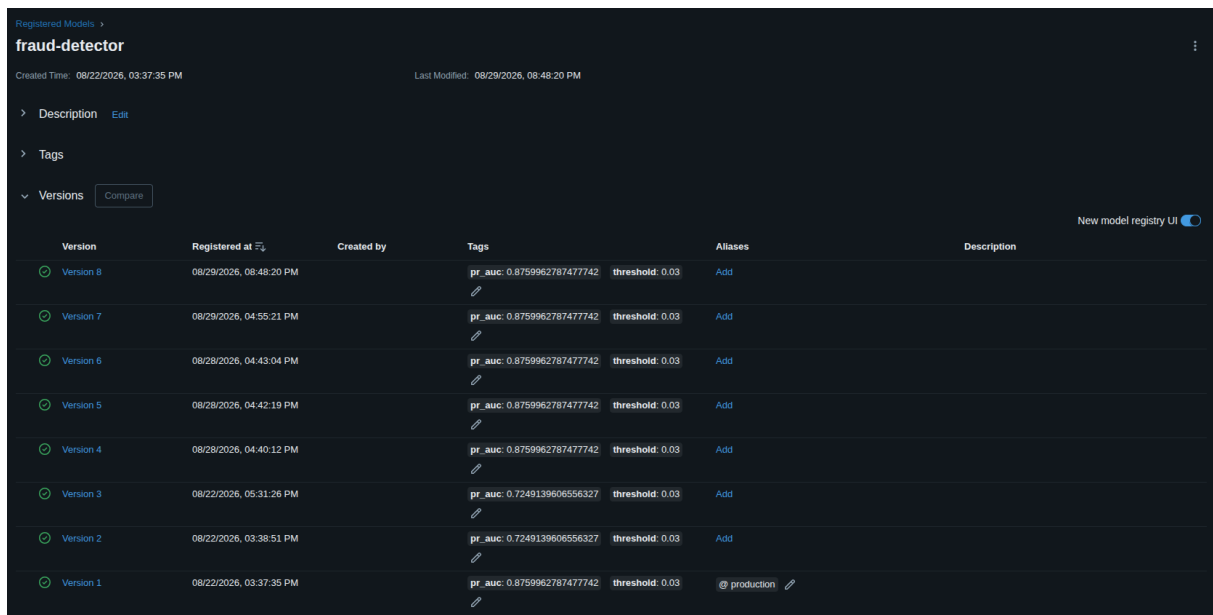


Figure 5: The outcome, in the Model Registry. Eight versions of **fraud-detector**, each tagged with the **pr_auc** and **threshold** it was registered under. The **@production** alias sits on Version 1 alone; versions 2–8 were registered and refused promotion, seven of them — including v8, created by the run above — because their PR-AUC did not *beat* the incumbent's.

8 What went wrong

A project’s failure record is more informative than its success record, because successes are over-determined and failures are specific. This section is complete rather than selective.

8.1 A deliberate failure: proving branch protection

Branch protection is a configuration setting, and configuration settings are believed rather than verified far too often. Phase 6 verified it.

Pull request #1 (`ci/verify-branch-protection`, “test: verify the model gate blocks a merge”) existed solely to fail. CI run 32718477985 is that engineered failure. With the required check red, GitHub reported the PR as `mergeStateStatus` `BLOCKED` and `merged=null` — it could not be merged. The PR was then closed, never merged.

ADR 0027 () records the resulting configuration, including a detail that would otherwise have cost an afternoon: the required check string is the *job name*: in the workflow file, not the job key and not the workflow name. All three differ, and only one of them matches.

8.2 An accidental failure: a commit split that broke CI

CI run 33247092413 was not deliberate. During Phase 8, the change was divided into commits in which `DriftDetectionError` was added by one commit while a different commit consumed it — and the producing commit did not land. `mypy -strict` caught it immediately:

```
src/monitoring/drift.py:53: error: Module "src.exceptions" has no attribute
"DriftDetectionError" [attr-defined]
```

Pull request #7 was closed rather than patched; the fix landed as 38c45a2 and PR #8 merged green. The lesson — *never split a prerequisite into a standalone commit its consumer needs* — was applied to every commit plan afterwards, visibly so in Phase 9, where the model loader, its tests and the artifact they load were deliberately kept in one commit.

It is worth noting what caught this. Not a test, not a review: a type checker, running in CI, against a commit that had passed every local check the author thought to run.

8.3 Two decision records disproved by their own verification

Twice, a record’s own verification step contradicted the record’s conclusion. Both were corrected in place rather than superseded.

adr 0025 () and the Compose project name (9f7cb1f). The record stated the mechanism correctly — Compose derives its project name from the compose file’s directory — and then drew the wrong conclusion, that a fresh clone therefore gets its own empty volume. It does not: because the compose file lives in `docker/`, the project name is always `docker` regardless of what the checkout is called, so a clone under `/tmp` resolved the *working environment’s populated volume*. The reproducibility test was silently testing nothing. It was re-run under an explicit `-p` flag, which the record now states as a requirement.

adr 0035 () and the ephemeral Prefect server (7c36cb7). The record claimed a flow run without `PREFECT_API_URL` never reaches the dashboard and that its database is destroyed at process exit. Both halves were wrong: Prefect’s ephemeral server is a temporary API process over the *same* `~/prefect/prefect.db`, and re-querying at closure found `excellent-chipmunk` — the very run that produced the claim — still present, `COMPLETED`, with all six of its log lines. The error was reading “Stopping temporary server” as data loss. The `Makefile` fix it had motivated stands, on a corrected justification.

8.4 A failed flow run, and the fix it forced

The Prefect database still holds `dexterous-porpoise`, state `FAILED`, created 2026-08-28 14:38:41 — the first orchestration attempt, before 9962a45 pointed the flows at the running server. It is the only failed flow run in eleven, and it was left in the history rather than cleaned up.

8.5 The Python 3.14 repin

Phase 0 pinned Python 3.14, the newest available. ADR 0004 () noted in its own trade-offs that a very new interpreter risks the ecosystem lagging behind it.

Phase 2 discovered exactly that: MLflow’s UI server imports `importlib.abc.Traversable`, removed in 3.14. Commit 54e22b9 repinned to 3.12.

The more interesting failure is what happened next: **the documentation did not follow the code**. For several commits the README’s stack table and ADR 0004 () both continued to claim 3.14, with ADR 0004 () additionally describing `requires-python = ">=3.12"` as a “known minor inconsistency” when reality was the reverse. The Phase 2 closure audit caught it (ec971f9), and ADR 0004 ()’s trade-off bullet now records the outcome instead of contradicting the repository.

8.6 The cold-start corrections

Phase 9 produced the same class of error twice, in the author’s own analysis rather than in the system.

Render documents a 15-minute idle spin-down. A 17-minute idle window was measured; the service answered in 0.449s, still warm. From that single observation two claims were written into the README and ADR 0043 (): that the service appeared never to sleep, and — reasoning onward — that an always-warm instance would consume roughly 730 of the 750 free monthly instance hours.

Longer windows demolished both. After ≈ 29 minutes idle the first request took **191 s**; after ≈ 3.4 hours, **111 s**.

Idle before request	First response
17 minutes	0.449 s (still warm)
≈ 29 minutes	191 s
≈ 3.4 hours	111 s
warm	0.30 s

The service does sleep; the wake takes two to three minutes; and it is not proportional to how long it slept. Both documents were corrected, and the README now *leads* its demo section with the wait — because a reviewer who hits an unexplained three-minute hang concludes the link is dead, which is the worst available outcome for the project’s most visible artifact.

The generalisable error is not about Render. It is that **a single negative observation was treated as evidence of absence**, and a short window was treated as representative of a long one.

8.7 A statistical test the repository named wrongly

The last episode was found by writing this paper, and could not have been found by running it.

Six places in the repository — `src/monitoring/drift.py`, `src/config.py`, `scripts/simulate_drift.py` and two decision records — stated that Evidently applied a **Kolmogorov–Smirnov** test to the thirty numerical features. Screenshotting the drift report for Figure 4 showed a **Stat Test** column reading **Wasserstein distance (normed)**. The report’s raw HTML settles it:

"stattest_name": "Wasserstein distance (normed)" appears **30 times**, and the string K-S appears **zero** times.

The cause was a correct observation over-generalised. ADR 0038 () had probed the installed release and recorded, accurately, `ValueDrift(column=f1, method=K-S p_value, threshold=0.05)`. But Evidently selects the per-column test by *reference sample size*, measured afterwards:

Reference rows	Test selected	Threshold
≤ 1000	K-S p-value	0.05
> 1000	Wasserstein distance (normed)	0.10

The probe used small synthetic frames and saw K-S. The **reference** DVC stage builds **5000** rows, so production runs Wasserstein.

Three things make this worth recording rather than quietly fixing. **It cost nothing at runtime**, precisely because of a decision made for an unrelated reason: the aggregate is located by `config.type` rather than by list index, so the extraction never depended on which test produced the number. **The reasoning it supported survives** — both tests compare distributions rather than summary statistics, so ADR 0040 ()'s correction about **Time** being bimodal still holds; only the name was wrong. And **it revealed a genuine test-coverage gap**: `tests/test_monitoring.py` uses 500-row frames, so the suite exercises the extraction and policy layers under K-S while production runs Wasserstein. That divergence is now documented in the docstring of `_drifted_share`.

All six sites were corrected, with dated correction notes added to ADR 0038 () and ADR 0040 () following the same in-place pattern the project used twice before.

8.8 What the record demonstrates

Five of these seven episodes were caught by an automated check or by a verification step designed to be capable of failing: a type checker in CI, a required status check, a reproducibility test run under a controlled project name, a re-query of live state at closure, a longer measurement window. The sixth — the stale Python version — was caught by a phase-closure audit whose explicit purpose was to look for that class of drift between documentation and reality.

The seventh was caught by nothing in the repository at all. No test could have failed on it, no type checker could have seen it, and the system behaved correctly throughout: the claim was in prose, and prose is not executed. It surfaced only because a figure for this paper required looking at the artifact rather than at the code that wrote it.

None of the seven were caught by reading the code carefully.

9 Limitations and future work

Each limitation below is recorded in the repository itself, not discovered for this paper.

The data remote is a local filesystem store. ADR 0005 () chose it as an interim measure, and it was never migrated. The consequence is precise: a clone on another machine cannot `dvc pull`, and must instead re-download the dataset from Kaggle and run `dvc repro`. The pipeline is reproducible; the *data transfer* is not. A shared bucket is the single change with the largest effect on third-party reproducibility.

No ground truth, therefore no concept drift. The system measures $P(X)$ and infers risk from it. A delayed-label pipeline — ingesting chargebacks weeks after the fact and measuring actual performance decay — is the natural next layer, and it is what would turn the monitoring from a proxy into a measurement.

No retrained model has ever been promoted. Retraining on unchanged data ties the incumbent and the gate refuses ties. The loop is proven; what remains undemonstrated is a promotion driven by genuinely new data, which requires a data source that actually moves.

The webhook alerting path has never reached a real endpoint. `DRIFT_WEBHOOK_URL` has only ever been exercised against `http://localhost:1`, which verifies the *failure* path — exception caught, logged, execution continuing — and nothing else. Logging always works and is tested; the delivery half is honestly unproven.

The measured drift share never reaches the Prefect dashboard. `detect_drift()` logs it at INFO on a module logger that propagates to a root logger left at WARNING, so the dashboard shows the boolean verdict but not the number behind it. This is a logging-configuration fix, deliberately not made during a phase closure.

The deployed model is frozen at build time. A promotion does not reach the public URL until the artifact is re-exported, committed and redeployed (ADR 0042 []). Acceptable for a demo, and explicitly not the architecture — the Compose topology retains the alias indirection.

The image ships roughly 400 MB it never uses. `nvidia-nccl-cu12`, a transitive dependency of XGBoost, in a CPU-only container — about 29% of the virtualenv. Removing it means constraining resolution in a way that also affects training, so it was recorded as a known cost rather than rushed.

The cost figures are illustrative. Every decision in the chain of Section 6.1 inherits that caveat, which is why they live in `params.yaml`.

The public deployment is slow to first response. Two to three minutes after an idle period, on a free tier that sleeps. Documented rather than paid for.

10 Lessons learned

10.1 Technical

Design for the phase that does not exist yet. The prediction log contract (Section 6.4) cost perhaps twenty minutes to specify in Phase 4 and saved Phase 8 from either a schema migration or a rewrite. The general form: when a component’s output will outlive the component, freeze its shape deliberately and write down that you have.

Build the loop inert, then energise it. Phase 7 constructed the entire retraining trigger — deployment resolution, fire-and-forget invocation, retry budgets — around a predicate hardcoded to `False`, and verified all of it. Phase 8 replaced the predicate and changed nothing else. Separating “is the wiring correct” from “is the signal correct” made each question answerable on its own.

Verification must be capable of failing. The most valuable checks in this project were the ones designed so that a specific mistake would surface: the engineered PR that proved branch protection blocks red builds; mutation testing that turned 11 green tests into 4 failures; a health check that inspects the response body because the status code cannot distinguish healthy from degraded. A check that cannot fail is decoration.

Measure the library, do not trust the documentation. Phase 8’s reference material prescribed an Evidently API that does not exist in the installed release. Writing extraction code against a probe of the actual version — and matching a stable identifier rather than a list index — was the difference between a loop that works and one that silently reads the wrong number.

A single negative observation is not evidence of absence. The cold-start error was not a carelessness about `Render`; it was a reasoning error about sampling, made twice, in the phase whose entire purpose was polish. Widening the window was the whole fix.

Centralised configuration has a visible, payable cost. Five commits exist solely to reconcile DVC hash drift caused by adding inert constants to `src/config.py`. The alternative — scattering configuration to avoid touching a declared dependency — is worse, but the cost is real and should be anticipated rather than discovered.

Write the decision down while you still remember the alternatives. The 44 records are useful mainly because they contain what was *rejected*. Reconstructed after the fact, they would have contained only what was chosen, which is the least interesting half.

10.2 Personal

This section is the author’s, in the first person, and is the only part of this paper not derived from repository evidence.

The hardest part was the prediction log. That is not the answer I would have predicted at the start — it is one file, appended to by one function, in a format with no schema. But it is the component that three separate phases all had to be right about simultaneously. Phase 4 had to choose a shape without knowing what would consume it. Phase 5 broke it in a way that produced no error at all: the container ran as a non-root user that could read `/app` and write nothing, so `mkdir` raised `EACCES`, the `except OSError` branch swallowed it exactly as designed, and every prediction was dropped while the endpoint kept answering 200. Nothing failed. The log was simply empty, and would have stayed empty until Phase 8 tried to read a distribution out of it. And Phase 8 then depended on the shape Phase 4 had guessed at four phases earlier.

A file that no test failed on, no exception surfaced from, and no dashboard reported — and which the entire monitoring half of the system was built on top of — turned out to be the thing that demanded the most care.

Containerisation was as straightforward as it looks. Phase 5 reads like an easy phase from the outside, and it was one. That is worth stating plainly rather than dramatising, because it was not luck: the environment was managed by `uv` with a committed lockfile from Phase 0,

the configuration was already centralised, and ADR 0024 ()’s single environment variable was the only thing that needed to change for the same code to talk to a containerised registry. The phase was easy because four earlier phases had made it easy.

I would keep the order. Nine gated phases, each closed before the next began, each with a checklist that had to pass. Looking back at it with the whole history visible, the sequencing is the part I would change least — data before training, training before registry, registry before serving, serving before containers, containers before CI, CI before orchestration, orchestration before monitoring. Every one of those dependencies turned out to be real, and several of them (Section 6.4) only revealed why they were real several phases later.

On why there are 44 decision records. Most people abandon that discipline early, and the honest reason I did not is twofold: so that anyone reading this can see what it actually takes to build a system like this, and so they can see how hard I tried. The records are 3,918 lines — more than the production code they explain — and a number of them document me being wrong and correcting it. I kept those in deliberately. A record that only contains decisions that turned out well is a record of a project that was never really examined.

11 Conclusion

The system described in this paper is not remarkable for its model. XGBoost at PR-AUC 0.876 on a well-studied public dataset is an unremarkable result, achieved in one phase out of nine, and the project spent no effort trying to improve it.

What the remaining eight phases produced is a model that can be *operated*: rebuilt from versioned inputs by a single command, promoted only when it beats the model already serving, resolved by role rather than by version number, served behind a typed boundary that logs everything it decides, shipped by a pipeline that refuses to deliver code failing lint, types, tests or a minimum quality bar, scheduled by an orchestrator with per-stage retry budgets, and watched by a loop that noticed a distribution change in its own traffic and retrained itself in thirty-three seconds without being asked.

The parts of that sentence which are genuinely hard are not the individual tools. They are the seams: that the preprocessor which produced training features is the same object that transforms a production request; that the log format chosen in Phase 4 was still the right one in Phase 8; that the retraining trigger built in Phase 7 needed no modification when the predicate behind it became real. Each of those is a decision made before its consequence was visible, written down at the time, and honoured later.

If there is a single transferable claim here, it is that the discipline is what produces the system, not the tooling. Any of these components could be swapped — Prefect for Dagster, Render for Cloud Run, Evidently for a hand-rolled two-sample test — without changing the project’s substance. What could not be swapped is the practice of writing down each decision with its alternatives, building verification that is capable of failing, and closing each phase against a checklist that was defined before the work started.

And the project’s most defensible result is one that looks, at a glance, like a failure: eight registered model versions, seven of them unpromoted, because a gate that requires strictly better performance kept refusing models that merely tied. That is the system declining to ship something it could not justify — which is, in the end, the whole point of building it this way.

12 Evidence index

Every substantive claim in this paper traces to one of the following sources, all re-queried on 30 August 2026 rather than carried forward from earlier work.

Source	Used for
<code>git log</code> , <code>git tag -l</code> <code>-n99</code>	Phase boundaries, commit counts, tag messages, the 142-commit / zero-merge-commit history
<code>docs/decisions/0001-0044</code>	Every design decision, alternative and trade-off cited
<code>reports/metrics.json</code>	PR-AUC 0.8760, recall 0.8878, precision 0.5541, F1 0.6824
<code>reports/validation.json</code>	284,807 rows, 492 fraud, rate 0.001727
MLflow <code>fraud-detection</code>	24 runs; the baseline/XGBoost/threshold comparison table experiment
MLflow Registry <code>fraud-detector</code> <code>params.yaml</code>	8 versions, <code>@production</code> on v1, 7 unpromoted Threshold 0.03, <code>scale_pos_weight</code> 577.29, cost parameters, expected-cost figures
<code>~/.prefect/prefect.db</code>	11 flow runs, both loop demonstrations, <code>dexterous-porpoise</code> FAILED
GitHub Actions API	33 CI runs (31/2), 15 CD runs (15/0), failure ids 32718477985 and 33247092413
GHCR registry API Live service	17 published image tags; public anonymous pull verified <code>/health</code> , <code>/docs</code> , <code>/model-info</code> , <code>/predict</code> responses and timings
Clean clone <code>project_context/</code>	<code>make setup + make test</code> = 83 passed in 13.17 s The prescribed roadmap, against which divergences were identified

Claims that could *not* be verified are marked as such in the text: the delivery half of the webhook alert, the branch-protection rule (the available API token lacks administrative scope), and the “key test” of Phase 9.

A The decision records

#	Decision	Phase
0001	Business metric: PR-AUC + recall priority	0
0002	False-negative vs false-positive cost asymmetry	0
0003	Dataset choice and known limitations	0
0004	Technology stack summary	0
0005	DVC remote: local filesystem store (interim)	1
0006	No data leakage: fit scaler on train split only	1
0007	Scale only Time and Amount , pass V1–V28 through	1
0008	Persist the fitted preprocessor (avoid training-serving skew)	1
0009	Stratified train/test split on the target	1
0010	Pandera contract: strict=True + lazy=True	1
0011	MLflow tracking backend: SQLite from day one	2
0012	Class-imbalance strategy: two levels, exclusive within the first	2
0013	Winning model: XGBoost over the logistic-regression baseline	2
0014	Operating threshold: cost-optimal (0.03), not F1-optimal	2
0015	Packaged model contract: artifacts injection, raw in, probability + decision out	3
0016	Promotion quality gate: PR-AUC alone, @production alone, strictly better	3
0017	Testing the Registry with hand-written doubles; evidencing it through the API	3
0018	Registry state at Phase 3 closure (dated snapshot)	3
0019	API response contract: probability alongside the decision	4
0020	API structure: inference in predict.py , model in app.state	4
0021	Prediction log as Phase 8’s contract; API testing strategy	4
0022	Container image: Python 3.12, non-root, body-matching health check	5
0023	Compose topology: two services, readiness gate, MLflow Host allowlist	5
0024	MLflow address from the environment, with a working local default	5
0025	Phase 5 verification strategy and the honest scope of the “key test”	5
0026	CI workflow shape: triggers, check order, enforced mypy --strict	6
0027	Branch protection as the enforcement boundary	6
0028	Model quality gate: MIN_PR_AUC = 0.75 and its derivation	6
0029	CD workflow, image tags, and public GHCR publication	6
0030	Phase 6 verification strategy, granularity, and DoD deviations	6
0031	prefect as a development dependency, not a production one	7
0032	mypy --strict extended to pipelines/ , and the serve() cast	7
0033	Orchestration design: task granularity, retries, no exception masking	7
0034	Testing Prefect flows without a server, and the gaps that leaves	7
0035	Phase 7 live verification: method, findings, and the ephemeral-server trap	7
0036	Phase 7 closure: verification by re-query, and what is deliberately unfinished	7

#	Decision	Phase
0037	Drift reference: training rows, raw space, frozen as a DVC stage output	8
0038	evidently as a production dependency, and reading its result by measurement	8
0039	Two-tier alerting, and why the drift report is not a dashboard module	8
0040	How the closed loop was demonstrated, and what it left behind	8
0041	Phase 8 closure: verification by rerun, and what the tag claims	8
0042	The public deployment carries its model inside the image	9
0043	Render, the free plan, and deploying the image CI already published	9
0044	Phase 9 closure: what the final tag claims, and what it does not	9

B Reproducing the results

From a clean clone, with only `uv` installed:

```
git clone https://github.com/adrianmarchramon/fraud-mlops-pipeline.git
cd fraud-mlops-pipeline
make setup          # uv sync + pre-commit install
make test           # 83 tests, no dataset required
```

The test suite passes without the dataset because `reports/metrics.json` and the exported model under `deploy/model/` are versioned in Git. To serve real predictions immediately, with no dataset and no MLflow:

```
MODEL_PATH=deploy/model uv run uvicorn src.api.main:app --port 8000
```

To rebuild the pipeline from raw data, a Kaggle API credential is required, because the DVC remote is a local store on the author's machine (ADR 0005 ()):

```
uv run kaggle datasets download -d mlg-ulb/creditcardfraud -p data/raw --unzip
uv run dvc repro      # validate -> preprocess -> train -> reference
```

To run the full system, including the drift loop, four processes are needed — the Prefect server, the deployment server, the API, and then the injection and the monitoring trigger:

```
make prefect-server  # dashboard on :4200
make prefect-serve   # serves both deployments
make serve           # the API on :8000
make simulate-drift  # POST 300 deliberately shifted transactions
make prefect-monitor # run the drift check now instead of waiting for 06:00
```

The public deployment requires no setup at all: <https://fraud-detection-api-unsm.onrender.com/docs>. Allow up to three minutes for the first response if the free-tier instance has spun down.